Dormitory Management System — Evaluation Report

Project: Student Dormitory Management System
Module: Advanced Web Development — University of Hildesheim
Branch Evaluated: Final-Requirements-Met
Report Date:2026-03-15

Evaluation Criteria

Metric	Classification Options	Description
Achievement	**Fully Achieved · Partially Achieved · Not Achieved**	Whether the requirement is met in the current implementation
Remove	**Should Remove · Can Remove · Shouldn't Remove**	Whether the implementation introduces overkill or scope creep beyond the requirement

1. Functional Requirements — Must

R01 — Student Registration

> *The system must provide registration functionality for students.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Backend endpoint POST /api/register exists and is functional. The Angular app has a /register route with a full RegisterComponent containing all required fields: name, email, password, confirm password, phone, student ID number, course, and university. The registration page calls AuthService.register() which hits the backend API. A link from the login page (goToRegister()) navigates to /register.
Remove	**Shouldn't Remove**	Core mandatory requirement, correctly implemented end-to-end.

R02 — Student Authentication & Login

> *The system must provide authentication and login functionality to students.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	/login page, AuthService, POST /api/login, JWT storage, and redirect to /student-dashboard all work end-to-end.
Remove	**Shouldn't Remove**	Core mandatory requirement, correctly implemented.

R02a — Admin Authentication & Login

> *The system must provide authentication and login functionality to dormitory administrators.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Same login flow as R02 with role-based redirect to /dashboard. JWT payload includes role and dormitory_id.
Remove	**Shouldn't Remove**	Core mandatory requirement, correctly implemented.

R03 — Online Dormitory Application Form for Students

> *The system must provide an online dormitory application form for students.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Student dashboard apply section has a working application form backed by POST /api/applications. The form auto-detects whether the student has an active contract and submits application_type: 'NEW' or 'EXTENSION' accordingly. Application history is displayed in a table.
Remove	**Shouldn't Remove**	Core mandatory requirement.

R04 — Student Personal Profile

> *The system must provide a personal profile for students.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Profile section in student dashboard with personal info (name, email, phone) and academic info (Student ID, course, university) backed by GET/PUT /api/profile. Profile JOINs dormitories and rooms to return live dormitory name, contact, room number, and floor.
Remove	**Shouldn't Remove**	Core mandatory requirement.

R05 — Admin Dashboard

> *The system must provide a dashboard for dormitory administrators.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Full admin dashboard with live stats cards (totalRooms, occupied, vacant, pending applications, open complaints, collection rate, overdue residents), activity feed, rooms, residents, payments, complaints, reports, and settings sections — all fetching from live API.
Remove	**Shouldn't Remove**	Core mandatory requirement.

R06 — Application Review for Admins

> *The system must provide application review functionality to dormitory administrators.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Admin residents section lists all applications in sub-tabs (New Applications, Extension Requests, Termination Requests) with applicant details and status badges. GET /api/applications is filtered by role — admins see all, students see their own.
Remove	**Shouldn't Remove**	Core mandatory requirement.

R07 — Application Acceptance for Admins

> *The system must provide application acceptance functionality to dormitory administrators.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Two-step acceptance flow: click Accept !’ select vacant room and fill contract details (start/end date, monthly rent, due day) !’ click "Confirm & Generate Contract" !’ a confirmation dialog overlay opens (acceptDialog) showing a summary !’ admin confirms !’ calls PATCH /api/applications/:id/status with a transactional room assignment and automatic contract + PDF generation.
Remove	**Shouldn't Remove**	Core mandatory requirement.

R07a — Application Rejection for Admins

> *The system must provide application rejection functionality to dormitory administrators.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Clicking Reject opens a rejectDialog modal overlay with three phases: remarks (enter rejection reason) !' confirm (summary preview with back button) !' result (success/failure feedback). Calls PATCH /api/applications/:id/ status with REJECTED status and optional remarks saved to DB.
Remove	**Shouldn't Remove**	Core mandatory requirement.

R08 — File Upload for Student Application Documents

> *The system must provide file upload functionality for students for submitting required documents.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	POST /api/applications/:id/files accepts up to 10 files (PDF/JPG/PNG, 5 MB each) via Multer middleware. Files are stored to /uploads/applications/ and recorded in application_files. GET /api/my-files and GET /api/files/:fileId/ download support retrieval. The student dashboard documents section is backed by these live endpoints, and the application form includes a document checklist with per-file upload controls. Client-side size validation (5 MB limit per file) is enforced before upload.
Remove	**Shouldn't Remove**	Core mandatory requirement, correctly implemented.

R09 — Digital Contract Creation for Accepted Students

> *The system must provide digital contract creation for accepted students.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	POST /api/contracts is called automatically during application acceptance. Contract records (start date, end date, monthly rent, due day, status ACTIVE) are stored in DB. A formal PDF is generated via generateContractPdf() (PDFKit) and saved to /uploads/contracts/.
Remove	**Shouldn't Remove**	Core mandatory requirement.

R10 — File Upload: Signed Rental Contract

> *The system must provide file upload functionality for students to upload signed rental contracts.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	POST /api/contracts/sign accepts a single PDF file (10 MB limit) via signedUpload Multer middleware. One-time-only upload enforced server-side. Client-side size validation (10 MB) rejects oversized files before the request is sent.

Remove	**Shouldn't Remove**	Explicitly stated as a mandatory file upload requirement, correctly implemented.

R11 — File Download: Student Contract

> *The system must provide file download functionality for students to download their uploaded contracts.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	GET /api/contracts/download streams the auto-generated contract PDF to the student browser. The endpoint checks that the requesting student owns the contract before serving the file. The student dashboard contract section has a Download button that activates once the contract PDF has been generated.
Remove	**Shouldn't Remove**	Explicitly stated as a mandatory file download requirement, correctly implemented.

R12 — File Upload: Monthly Rent Payment Receipts

> *The system must provide file upload functionality for students for uploading monthly rent payment receipts.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	POST /api/payments accepts a receipt file attachment via receiptUpload Multer middleware (5 MB limit), stored to /uploads/receipts/. Receipt path is saved to the rent_payments table. Client-side size validation (5 MB) rejects oversized files before upload.
Remove	**Shouldn't Remove**	Mandatory requirement, correctly implemented.

R13 — Storage of Rent Payment Records

> *The system must provide storage of rent payment records for students.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	POST /api/payments stores payment in DB. GET /api/payments retrieves history with status badges. Student payment history table is displayed in the student dashboard. Overdue tracking via GET /api/payments/overdue supplements this.
Remove	**Shouldn't Remove**	Core mandatory requirement.

R14 — File Download: Rent Payment Records as PDF

> *The system must provide file download functionality for students to download rent payment records as a PDF document.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	GET /api/reports/:id/download streams an actual PDF generated by generatePdfReport() (PDFKit). The report includes occupancy stats, payment summary, complaint list, and resident info. Download is triggered from the student dashboard after report generation completes (via the report progress modal).
Remove	**Shouldn't Remove**	Mandatory requirement, correctly implemented with real PDF content.

R15 — Contract Extension for Admins

> *The system must provide contract extension functionality to dormitory administrators.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Contract extension is handled via the standard application workflow: students submit an application_type: 'EXTENSION' application, which the admin reviews in the dedicated Extension Requests sub-tab of the residents section. The admin uses the same Accept flow (room + dates + rent !' confirmation dialog !' contract update) to approve extensions. The DB enum supports EXTENDED status.
Remove	**Shouldn't Remove**	Core mandatory requirement, fully implemented via the extension application workflow and sub-tab.

R16 — Contract Termination for Admins

> *The system must provide contract termination functionality to dormitory administrators.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Two termination paths exist. (1) Student-initiated: students submit a termination request via POST /api/termination-requests, visible in the admin's Termination Requests sub-tab; admin can Accept or Reject directly from the table. (2) Admin-initiated: a "Contract Termination" button opens an adminTerminateDialog modal — admin selects a student with an active contract, enters a reason, reviews a summary, and confirms via POST /api/contracts/admin-terminate. Both paths mark the contract TERMINATED, set the end date, and free the room.
Remove	**Shouldn't Remove**	Core mandatory requirement, fully implemented via both student-initiated and admin-initiated termination flows.

R17 — Backend Long-Running Report Generation

> *The system must provide a backend process to generate a rent payment history report (mandatory long-running task).*

Metric	Classification	Justification
Achievement	**Fully Achieved**	POST /api/reports inserts a progress record (status PENDING, 0%) then delegates actual generation to setImmediate(() => generatePdfReport(...)) — a non-blocking async call. generatePdfReport() executes multi-stage DB queries, builds a PDFKit document, writes to disk, and updates the progress table at 25%, 50%, 75%, and 100% milestones.
Remove	**Shouldn't Remove**	One of the three explicitly highlighted mandatory technical patterns, correctly implemented.

R18 — Progress Status Information During Report Generation

> *The system must provide progress status information to students during the generation of rent payment reports.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	GET /api/reports/:id/progress reads the progress table and returns { percentage, reportStatus }. The frontend polls this endpoint every 600 ms via ProgressService.startPolling(), updates the modal state, and auto-stops polling when status reaches COMPLETED, CANCELLED, or FAILED.
Remove	**Shouldn't Remove**	Explicitly marked as a mandatory progress display requirement, correctly implemented.

R19 — Visual Progress Indicator During Backend Processing

> *The system must provide a visual progress indicator to students while backend processing is running.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	The student dashboard opens a reportModal on generation start. ProgressComponent renders a live progress bar bound to reportModal.percentage, a status label, a Cancel button, and a Download button that activates when status is COMPLETED. The bar reflects true backend progress polled from the progress table via ProgressService.
Remove	**Shouldn't Remove**	Companion to R17 and R18; all three are implemented together correctly.

R20 — Complaint Submission for Students

> *The system must provide complaint submission functionality for students.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Student dashboard complaints section has a form (title + description). Clicking "Submit" triggers the complaint confirmation dialog (R22) before the actual API call. Submitted complaints appear in the history list with status badges.
Remove	**Shouldn't Remove**	Core mandatory requirement.

R21 — Complaint Management for Admins

> *The system must provide complaint management functionality to dormitory administrators.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Admin requests section lists all complaints with status badges and workflow buttons (SUBMITTED !' IN_PROGRESS !' RESOLVED) backed by PATCH /api/complaints/:id/status.
Remove	**Shouldn't Remove**	Core mandatory requirement.

R22 — Dialog Window: Student Complaint Confirmation

> *The system must provide a dialog window for students to confirm complaint submission (mandatory dialog requirement).*

Metric	Classification	Justification
Achievement	**Fully Achieved**	A complaintDialog modal overlay is implemented in student-dashboard.html. When the student clicks Submit in the complaint form, openComplaintDialog() validates input and opens a conf-backdrop / conf-modal overlay. The dialog has two phases: confirm (shows title and description preview with Cancel / Submit Complaint buttons) and result (success or failure feedback). The actual submitComplaint() API call is only made after the student confirms inside the dialog.
Remove	**Shouldn't Remove**	Explicitly flagged as fulfilling the mandatory dialog window requirement. Correctly implemented.

R23 — Dialog Window: Admin Application Confirmation

> *The system must provide a dialog window for dormitory administrators to confirm application acceptance or rejection.*

Metric	Classification	Justification

Achievement	**Fully Achieved**	Two separate modal overlay dialogs are implemented in dashboard.html. Accept dialog (acceptDialog): renders as a conf-backdrop / conf-modal overlay with a contract details summary (student, room, dates, rent, due day), Cancel and Confirm buttons, and a result phase. Reject dialog (rejectDialog): has three phases — remarks (enter rejection reason), confirm (summary preview), and result (success/error feedback).
Remove	**Shouldn't Remove**	Companion to R22; both implemented as proper dialog window overlays.

R24 — Menu-Based Navigation Using Angular Routing

> *The system must provide menu-based navigation for all users using Angular routing (mandatory router requirement).*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Top-level Angular routing is implemented (/, /login, /register, /dashboard, /student-dashboard). All section navigation within both dashboards uses Angular child routes. Sidebar items trigger router.navigate(['/dashboard', id]). The URL updates to /dashboard/rooms, /dashboard/payments, etc. when a section is selected. activeNav is synced from the router via NavigationEnd events rather than direct mutation.
Remove	**Shouldn't Remove**	Explicitly highlighted as fulfilling the mandatory router-based menu requirement. Fully implemented.

R25 — Role-Based Access Control: Students

> *The system must provide role-based access control for students.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Server-side: requireRole('STUDENT') guards student-only endpoints. Client-side: authGuard (redirects unauthenticated users to /login) and roleGuard (redirects wrong-role users to their correct dashboard) are implemented in src/app/guards/ and applied to /student-dashboard via canActivate: [authGuard, roleGuard] with data: { role: 'STUDENT' }.
Remove	**Shouldn't Remove**	Mandatory requirement, fully implemented on both frontend and backend.

R25a — Role-Based Access Control: Admins

> *The system must provide role-based access control for dormitory administrators.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Server-side: requireRole('ADMIN') guards admin endpoints. Client-side: same authGuard and roleGuard are applied to /dashboard via canActivate: [authGuard, roleGuard] with data: { role: 'ADMIN' }. An admin visiting /student-dashboard is redirected to /dashboard and vice versa.

Remove	**Shouldn't Remove**	Mandatory requirement, fully implemented on both frontend and backend.

2. Functional Requirements — Should

R26 — Student Dashboard: Application Status Overview

> *The system shall provide an overview dashboard for students showing application status.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Student dashboard overview cards and the apply section both show application status (PENDING/ACCEPTED/REJECTED) fetched from GET /api/applications.
Remove	**Shouldn't Remove**	Useful student-facing feature matching the requirement.

R26a — Student Dashboard: Contract Status Overview

> *The system shall provide an overview dashboard for students showing contract status.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Student overview shows a "Contract" stat card and a dedicated contract section displaying status badge, start/end dates, monthly rent, due day, next payment info, overdue warning, and contract signing/download controls. The termination request status is also shown inline. All data is live from GET /api/contracts.
Remove	**Shouldn't Remove**	Useful feature correctly matching the requirement.

R27 — Admin Dashboard: Pending Applications Overview

> *The system shall provide an overview dashboard for dormitory administrators showing pending applications.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Admin overview shows a "Pending Apps" count card derived from GET /api/stats. The activity feed highlights recent applications. A quick-action button navigates directly to the residents section.
Remove	**Shouldn't Remove**	Correctly implemented.

R28 — Complaint Status Visibility for Students

> *The system shall provide the ability to view complaint status for students.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Student dashboard complaints section shows each complaint with status badge (SUBMITTED / IN_PROGRESS / RESOLVED) from GET /api/complaints.
Remove	**Shouldn't Remove**	Correctly implemented.

R29 — Filter Rent Payments by Month for Students

> *The system shall provide the ability to filter rent payments by month for students.*

Metric	Classification	Justification
Achievement	Partially Achieved	The payment submission form has a month dropdown (paymentMonth) used to select which month a payment covers. However, this dropdown is for submission, not for filtering the payment history list. No separate filter, search input, or client-side filtering exists to narrow the displayed payment history by a selected month.
Remove	Can Remove	A "Should" requirement. A simple client-side filter would suffice if time permits; it is not critical.

R30 — Basic Validation of Uploaded Documents

> *The system shall provide basic validation of uploaded documents (file type and size).*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Server-side validation is enforced by Multer middleware (PDF/JPG/PNG, 5 MB per file; contract 10 MB). Client-side: all four upload handlers (onChecklistFileSelect, onExtraFileSelect, onSignedContractSelect, onReceiptSelect) validate file.size before assigning the file. Oversized files are rejected immediately in the browser with a specific error message naming the offending file. The accept=".pdf,.jpg,.jpeg,.png" attribute on <input type="file"> provides browser-level type filtering.
Remove	**Shouldn't Remove**	"Should" requirement, fully met.

R31 — Notification Messages After Successful Actions

> *The system shall provide notification messages for students after successful actions.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Both dashboards show inline success/error alert boxes after API operations complete. Dialog result phases also display outcome messages (e.g. "Complaint Submitted!", "Application Accepted!", "Contract Terminated"). Real-time bell notifications via SSE and toast pop-ups are implemented for both admin and student dashboards.
Remove	**Shouldn't Remove**	Good UX and a "Should" requirement that is met.

3. Functional Requirements — Can

R32 — Search Functionality for Admins (Tenant Records)

> *The system can provide search functionality for dormitory administrators for tenant records.*

Metric	Classification	Justification
Achievement	**Not Achieved**	No search box or filtering input exists in the admin dashboard for tenants, applications, payments, or complaints.
Remove	Can Remove	**Optional "Can" requirement. Not implemented; not a priority.**

R33 — Data Export for Admins

> *The system can provide data export functionality for dormitory administrators.*

Metric	Classification	Justification
Achievement	**Not Achieved**	No CSV/Excel export functionality exists. Admin reports are PDF-based; no tabular data export is provided.
Remove	Can Remove	Optional "Can" requirement. Not a priority.

R34 — Profile Update for Students

> *The system can provide profile update functionality for students.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	GET/PUT /api/profile is fully implemented. The student dashboard profile section allows editing personal info (name, email, phone) and academic info (Student ID, course, university), saving to DB.

Remove	**Shouldn't Remove**	Already implemented and adds value.

4. Technical Requirements (Not Counted for Grading)

TR01 — Angular Frontend with Router-Based Navigation

> *The system must be implemented using an Angular frontend with router-based navigation.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Angular 21 is used throughout. Top-level routing is implemented with five routes. In-dashboard section navigation now uses Angular child routes (/dashboard/rooms, /dashboard/payments, etc.) with DashboardSectionComponent as the child. authGuard and roleGuard are applied. The sidebar calls router.navigate() and activeNav is synced from ActivatedRoute via NavigationEnd events.
Remove	N/A	Technical requirement.

TR02 — Backend Technology

> *The system must utilize backend technology to support the frontend.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Express.js + Node.js backend (server.js) with MySQL2, JWT, bcrypt, Multer, and PDFKit, serving 40+ REST endpoints with full auth and role middleware.
Remove	N/A	Technical requirement.

TR03 — Automated Tests with Karma

> *The system must provide automated tests executed with Karma.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	The project now uses Karma with karma-jasmine and karma-chrome-launcher. A full test suite of 418 tests spans all services (AuthService, ApiService, NotificationService, ProgressService), both dashboard components (DashboardComponent, StudentDashboardComponent), route guards (AuthGuard, RoleGuard), and E2E integration tests (app.e2e.spec.ts). All 418 tests pass. The previous Vitest toolchain has been replaced with Karma.
Remove	N/A	**Technical requirement. Fully achieved.**

TR04 — Test Coverage Above 60% (Including E2E)

> *The system must achieve test coverage above 60%, including e2e tests.*

Metric	Classification	Justification
Achievement	**Fully Achieved**	Istanbul/karma-coverage reports coverage at all four metrics exceeding 80%: Statements 85.33%, Branches 80.28%, Functions 84.98%, Lines 86.61%. All four metrics exceed the 60% threshold stated in the requirement (and exceed the 80% internal target applied individually to line, branch, and function coverage). E2E tests in app.e2e.spec.ts are included in the test run and contribute to coverage.
Remove	N/A	**Technical requirement. Fully achieved with margin above threshold.**

TR05 — Runnable VirtualBox VM Delivery by 2026/03/17

> *The system must be delivered as a runnable VirtualBox VM including documentation until 2026/03/17.*

Metric	Classification	Justification
Achievement	Not Assessed	Deadline is 2026-03-17. VM preparation and documentation packaging status not evaluated in this report.
Remove	N/A	Technical requirement.

5. Over-Engineering Analysis (Beyond Requirements)

The following features were implemented beyond what the requirements specify. Each is evaluated for whether it should be removed to simplify the codebase.

Multi-Dormitory Support

What was added:dormitoryid foreign key on users, adminprofiles, dormapplications, contracts, complaints, rentpayments, reports, and dormitories table. All admin API queries filter by req.user.dormitoryid. JWT payload includes dormitoryid.

Metric	Classification	Justification
Remove	**Should Remove**	The requirements describe a system for one dormitory with two roles. Multi-dormitory support complicates every API query, the data model, JWT, and seeding. None of the requirements mention multiple dormitories. This should be stripped back to a single-dormitory model.

Admin Settings: Dormitory Info Editing

What was added:A full dormitory info form in the settings section (name, address, contact email/phone, max capacity) backed by PUT /api/admin/profile.

Metric	Classification	Justification
Remove	**Should Remove**	No requirement asks for dormitory metadata management. This exists only as a consequence of the multi-dormitory over-engineering above.

Payment Verification Workflow

What was added:verificationstatus column on rentpayments (PENDING_VERIFICATION / VERIFIED / REJECTED). PATCH /api/payments/:id/verify and PATCH /api/payments/:id/reject endpoints. Admin dashboard includes verify/reject buttons and receipt download.

Metric	Classification	Justification
Remove	Can Remove	R12 requires only that receipts can be uploaded. R13 requires storage. No requirement asks for admin payment verification. The feature adds genuine value but is beyond the stated scope.

Room Management Grid (Admin)

What was added:A full rooms grid section in the admin dashboard with floor grouping, status filtering, room cards with resident names, and maintenance-clearing dialog.

Metric	Classification	Justification
Remove	Can Remove	Rooms appear in the requirements only as a detail of application acceptance (room assignment). A dedicated room management grid is not required. However, GET /api/rooms/vacant is needed for the acceptance dropdown, so the endpoint must remain.

PDFKit Contract with Formal Legal Clauses

What was added:generateContractPdf() produces a multi-page formal contract with header, tenant/dorm info block, rental terms, payment schedule, due-day clause, and termination clause.

Metric	Classification	Justification
Remove	Can Remove	R09 asks only for "digital contract creation" and R11 for downloadability. The PDFKit dependency is necessary for R14/R17 anyway, but the full legal-clause template is beyond the requirement.

Overdue Tracking and Collection Rate Stats

What was added:GET /api/payments/overdue calculates overdue months. GET /api/stats returns paidThisMonth, totalOutstanding, overdueResidentCount, and collectionRate. Student dashboard shows next payment info and overdue

warnings.

Metric	Classification	Justification
Remove	Can Remove	No requirement specifies overdue tracking, collection rate calculation, or next-payment notifications. Useful business features but exceed the stated scope.

Notification System (SSE + Bell + Toasts)

What was added:A full real-time notification system using Server-Sent Events (SSE): notifications DB table, pushNotification() helper, GET /api/notifications (load), PATCH /api/notifications/:id/read (mark read), GET /api/events (SSE stream). Both admin and student dashboards have a bell icon with badge, dropdown panel, and toast pop-ups. Notification preferences (enable/disable per category) stored in the dormitories table.

Metric	Classification	Justification
Remove	Can Remove	R31 requires only "notification messages after successful actions" (i.e., inline success/error feedback). A full SSE-powered real-time notification system is substantially beyond R31. Consider simplifying to inline messages if codebase size is a concern.

Student Dashboard: "Important Contacts" Card

What was added:A card in the student overview section showing dorm admin phone, maintenance number, emergency contact, and office hours. Two fields remain hardcoded.

Metric	Classification	Justification
Remove	Can Remove	No requirement specifies a contacts card. As a partially hardcoded element it adds no real value.

Termination Request Workflow (Student-Initiated)

What was added:A multi-phase terminationDialog modal in the student contract section lets students submit a termination request (reason + requested end date) via POST /api/termination-requests. Admin sees requests in a dedicated "Termination Requests" sub-tab.

Metric	Classification	Justification
Remove	Can Remove	R16 only requires that admins can terminate contracts. However, since the feature is tightly integrated with R16's full implementation (admin reviews and approves/rejects these requests), removing it would complicate R16. Consider keeping it as it directly supports R16 fulfillment.

Admin Report Generation (Chart.js + PDFKit)

What was added: Admin dashboard has a "Generate Report" button that renders three Chart.js charts (occupancy doughnut, finances bar, maintenance doughnut) on hidden canvases, extracts base64 PNG, and sends to POST /api/admin/reports where PDFKit embeds the images into a formatted admin report PDF.

Metric	Classification	Justification
Remove	Can Remove	R17 specifies the student-facing rent payment report. An admin-facing chart PDF report is an extra. However, the feature demonstrates good use of the required technology stack (PDFKit, long-running task).

Academic Info Fields in Student Profile

What was added:studentidnumber, course, and university fields in student_profiles, with corresponding form fields in the student dashboard profile section.

Metric	Classification	Justification
Remove	Can Remove	R04 asks only for "a personal profile for students". Academic fields are extras. Minimal complexity and harmless, but strictly speaking beyond the requirement.

6. Summary Scorecard

Achievement Summary

Category	Fully Achieved	Partially Achieved	Not Achieved
Must (R01–R25a)	26	0	0
Should (R26–R31)	5	1	0
Can (R32–R34)	1	0	2
Technical (TR01–TR05)	4	0	0 + 1 not assessed
Total	36	1	2 + 1 N/A

Progress Since Previous Branch (Achieve-all-functional-requirements)

Previously Not Achieved !' Now Fully Achieved
TR03 — Automated Karma tests (418 tests; full Karma + karma-jasmine + karma-chrome-launcher setup)
TR04 — Test coverage above 60% (statements 85.33%, branches 80.28%, functions 84.98%, lines 86.61%)

Remaining Gaps

ID	Status	Requirement
R29	Partially	Month dropdown exists for payment submission but not for filtering payment history

TR05	Not Assessed	VM delivery deadline: 2026-03-17

Remove Summary

Classification	Count	Key Items
Should Remove	2	Multi-dormitory support, dormitory info editing
Can Remove	9	Payment verification, room grid, PDFKit contract clauses, overdue tracking, notification system (SSE), Important Contacts card, student termination request workflow, admin chart report, academic profile fields
Shouldn't Remove	All core features	Authentication, registration, applications, file uploads, contracts, complaints, payments, reports, profiles, dialog overlays, child routes, route guards

7. Priority Recommendations

Based on the analysis, the following actions are recommended in priority order:

Immediate (Remaining mandatory gap):

- Prepare VirtualBox VM with full setup instructions — covers TR05

Short-term (Partial implementations to complete):

- Add month filter to student payment history list — covers R29

Clean-up (Remove scope creep):

- Strip multi-dormitory support back to single-dormitory model — simplifies entire codebase
- Remove or fully dynamicize Important Contacts card (hardcoded phone numbers)